

Git 常用命令指南

目录

- [基础配置](#)
- [仓库操作](#)
- [文件操作](#)
- [分支管理](#)
- [合并与冲突](#)
- [远程操作](#)
- [撤销与重置](#)

0、基本演示

创建文件夹，git init;

创建test1.txt，写入"hello w0rld"，git add，git commit;

```
• (base) PS C:\Users\86158\Desktop\learning_git> git init
Initialized empty Git repository in C:/Users/86158/Desktop/learning_git/.git/
• (base) PS C:\Users\86158\Desktop\learning_git> git add .
• (base) PS C:\Users\86158\Desktop\learning_git> git commit -m "提交test1.txt"
[master (root-commit) 8af2d6c] 鎖愼氮test1.txt
1 file changed, 1 insertion(+)
create mode 100644 test1.txt
```

连接远程仓库，push本地master分支;

```
• (base) PS C:\Users\86158\Desktop\learning_git> git remote add origin https://github.com/s1mple555/git_learning.git
• (base) PS C:\Users\86158\Desktop\learning_git> git remote
origin
• (base) PS C:\Users\86158\Desktop\learning_git> git push origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 231 bytes | 231.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/s1mple555/git_learning.git
* [new branch]      master -> master
```

创建本地debug分支，在分支中修改test1.txt中内容为"hello world";

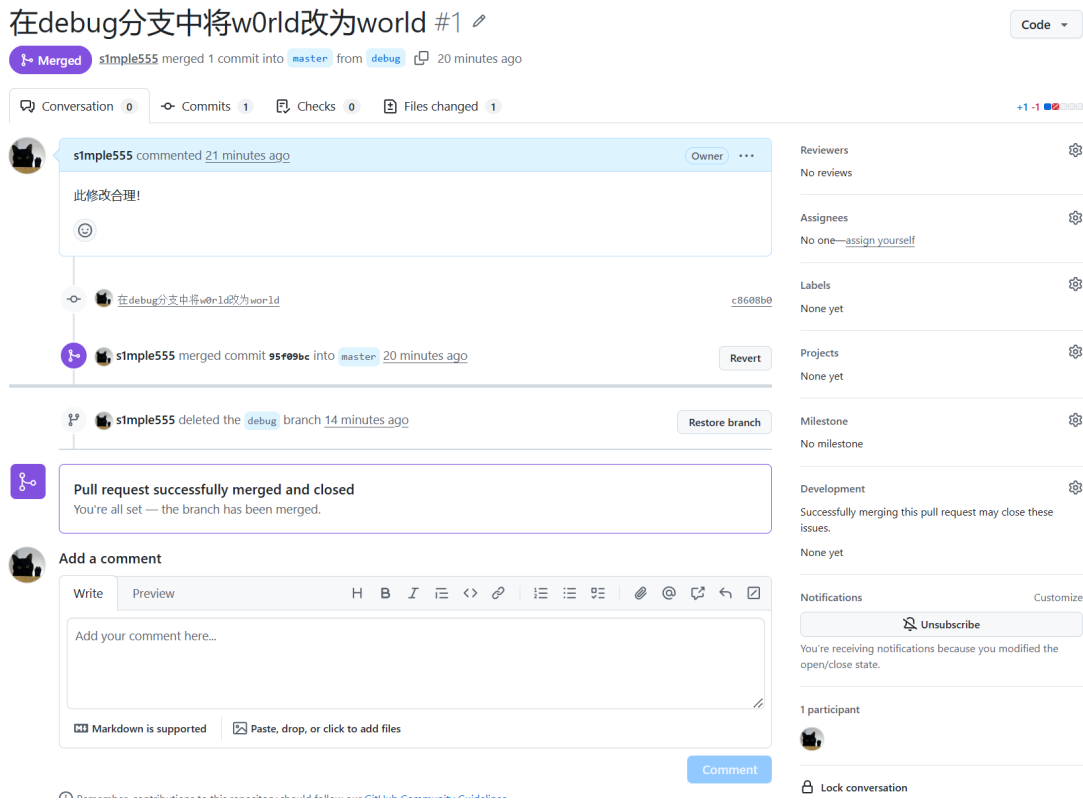
```
• (base) PS C:\Users\86158\Desktop\learning_git> git branch debug
• (base) PS C:\Users\86158\Desktop\learning_git> git branch
debug
* master
• (base) PS C:\Users\86158\Desktop\learning_git> git checkout debug
Switched to branch 'debug'
• (base) PS C:\Users\86158\Desktop\learning_git> git add .
• (base) PS C:\Users\86158\Desktop\learning_git> git commit -m "在debug分支中将w0rld改为world"
[debug c8608b0] 鎚 | ebug錄喫敲涓涓w0rld鎚逛负world
1 file changed, 1 insertion(+), 1 deletion(-)
```

把本地的debug分支推到仓库的debug分支（仓库创建debug分支）;

```
• (base) PS C:\Users\86158\Desktop\learning_git> git push origin debug
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 288 bytes | 288.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'debug' on GitHub by visiting:
remote:      https://github.com/s1mple555/git_learning/pull/new/debug
remote:
To https://github.com/s1mple555/git_learning.git
* [new branch]      debug -> debug
```

随后，在github中的pull request栏，提起PR，并进行合并；

合并之后远程仓库中的test1.txt内容变为hello world；



debug分支完成了使命，这里删除远程仓库的debug分支和本地的debug分支；

```
(base) PS C:\Users\86158\Desktop\learning_git> git push origin --delete debug
To https://github.com/s1mple555/git_learning.git
- [deleted]          debug
```

```
• (base) PS C:\Users\86158\Desktop\learning_git> git branch -d debug
Deleted branch debug (was c8608b0).
• (base) PS C:\Users\86158\Desktop\learning_git> git branch
* master
• (base) PS C:\Users\86158\Desktop\learning_git> git branch -r
origin/master
○ (base) PS C:\Users\86158\Desktop\learning_git> █
```

切换到本地仓库的master分支，git pull，本地仓库master分支与远程仓库进行合并（从w0rld到world）；

```
(base) PS C:\Users\86158\Desktop\learning_git> git pull origin master
From https://github.com/s1mple555/git_learning
* branch          master      -> FETCH_HEAD
Updating 8af2d6c..95f09bc
• Fast-forward
  test1.txt | 2 + -
  1 file changed, 1 insertion(+), 1 deletion(-)
(base) PS C:\Users\86158\Desktop\learning_git> git push origin --delete debug
To https://github.com/s1mple555/git_learning.git
- [deleted]          debug
(base) PS C:\Users\86158\Desktop\learning_git> █
```

随后，我在远程仓库（github网页端）添加了readme文件；

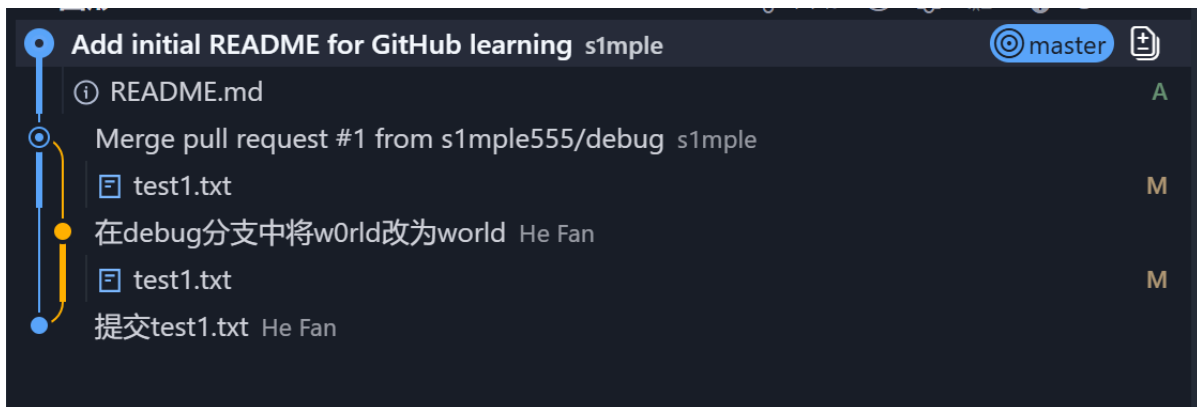
在本地的master分支下拉取，这次不使用git pull命令，使用git fetch+git merge命令；

(这个命令组合不常用，一般都是git pull)

```
(base) PS C:\Users\86158\Desktop\learning_git> git fetch origin
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1010 bytes | 126.00 KiB/s, done.
From https://github.com/s1mple555/git_learning
95f09bc..5fd8987 master -> origin/master
(base) PS C:\Users\86158\Desktop\learning_git> git branch
* master
(base) PS C:\Users\86158\Desktop\learning_git> git merge origin/master

Updating 95f09bc..5fd8987
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
(base) PS C:\Users\86158\Desktop\learning_git>
```

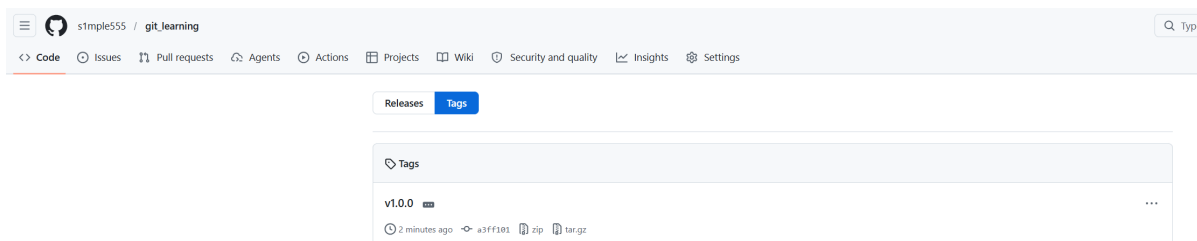
此时，左侧的图形化界面：蓝色表示普通提交，橙色代表合并，其它颜色代表不同的分支；（这里没有云朵的话，git push -u一次，建立连接即可）

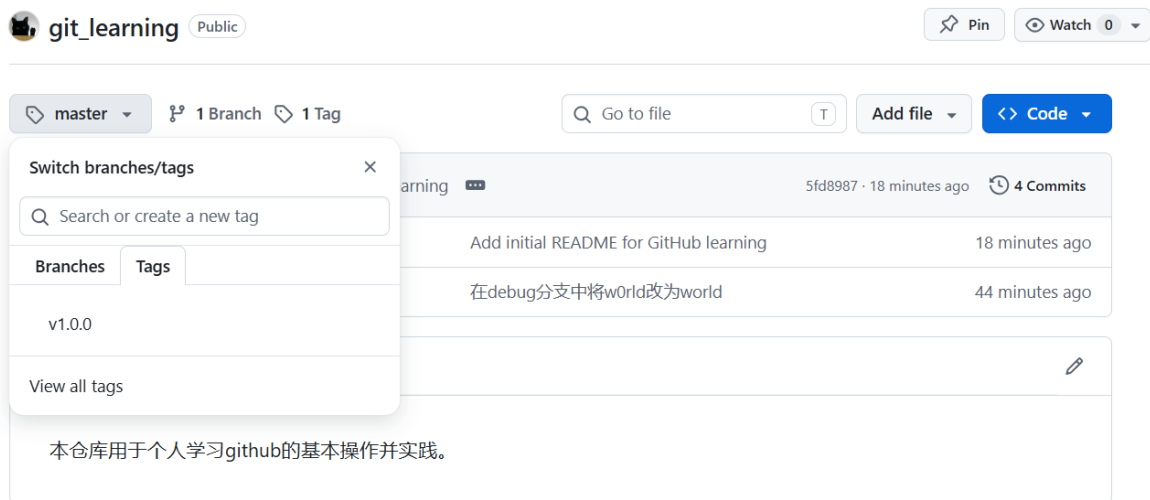


我创建了tag.txt，里面标识了v1.0.0，将其commit，此时创建tag，push到远程仓库的v1.0.0分支，作为一个版本分支；

此时仓库右侧的release中出现了tag为v1.0.0的版本，也在分支里多了一个新tag；

```
(base) PS C:\Users\86158\Desktop\learning_git> git add .
(base) PS C:\Users\86158\Desktop\learning_git> git commit -m "创建版本文件，标识v1.0.0"
[master a3ff101] 錄涇緩鏑塚滄鏑困欢鏑岋燻璇啾1.0.0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 tag.txt
(base) PS C:\Users\86158\Desktop\learning_git> git tag v1.0.0
(base) PS C:\Users\86158\Desktop\learning_git> git tag
v1.0.0
(base) PS C:\Users\86158\Desktop\learning_git> git push origin v1.0.0
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 342 bytes | 342.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/s1mple555/git_learning.git
 * [new tag]          v1.0.0 -> v1.0.0
```





变基的使用场景是什么：开发功能，此时你可以在获取最新的master分支，并git merge feature。此时也可以使用rebase变基。

场景：开发一个新功能

PlainText

- 1 初始状态：
- 2 master —A———B

你在 feature 分支开发了 3 个提交：

PlainText

- 1 master —A———B
- 2 \
- 3 feature C———D———E

同时，同事在 master 上也提交了新代码：

PlainText

- 1 master —A———B———F
- 2 \
- 3 feature C———D———E

方式 2：使用 Rebase

\$ Bash

- 1 git checkout feature
- 2 git rebase master

结果：

PlainText

- 1 master —A———B———F———C'———D'———E'
- 2 (feature)

1、基础配置

```
# 下载git
git --version
# 配置用户信息
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# 查看配置
git config --list

# 查看特定配置
git config user.name
```

2、仓库操作

初始化操作，这里与IDE中左侧初始化本地仓库是一致的。

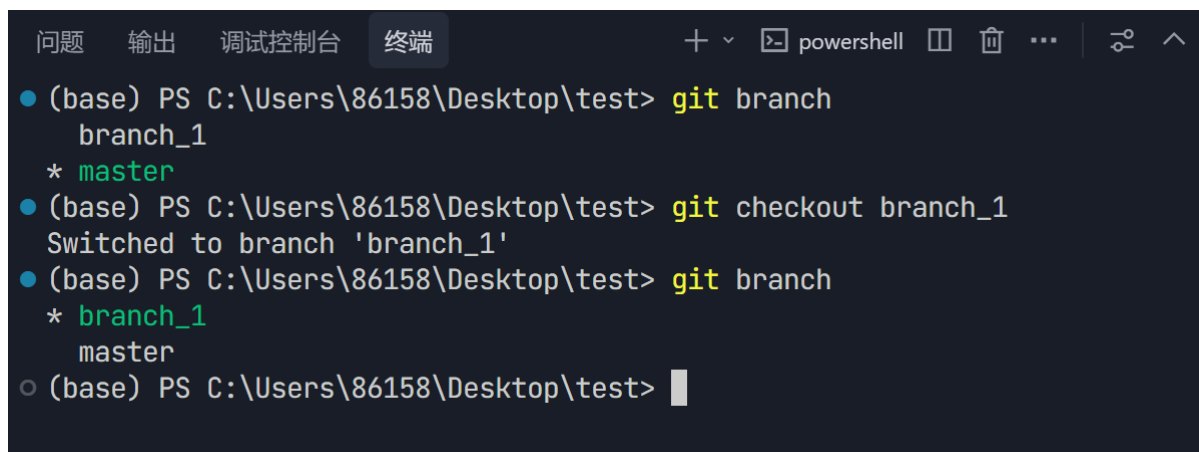
```
# 初始化新仓库
git init

# 克隆远程仓库
git clone <repository-url>

# 查看仓库状态（本地与远程仓库同步状态、未暂存、已暂存的文件）
git status

# 查看当前分支
git branch

# 切换分支(主分支为master)
git checkout branch_name
```



The screenshot shows a PowerShell terminal window with the title bar 'powershell'. The terminal output is as follows:

```
● (base) PS C:\Users\86158\Desktop\test> git branch
    branch_1
    * master
● (base) PS C:\Users\86158\Desktop\test> git checkout branch_1
Switched to branch 'branch_1'
● (base) PS C:\Users\86158\Desktop\test> git branch
    branch_1
    * master
○ (base) PS C:\Users\86158\Desktop\test> 
```

3、文件操作

```
# 添加文件到暂存区
git add <file>          # 添加指定文件
git add .                # 添加所有文件

# 提交更改
git commit -m "提交信息"

# 删除文件
git rm <file>            # 删除文件并提交，文件不再被 Git 版本控制，工作区中的文件也被删除，
                        # 需要git commit执行这个删除操作，磁盘上和git上都删除
git rm --cached <file>   # 仅从 Git 追踪中删除，保留文件在磁盘上
```

4、分支管理

本地分支：用于便于代码管理，创建个分支，完成任务后，合并到master

远程分支：远程仓库的分支，由某用户push自己的本地仓库分支上去

用户push了分支上去，可以申请PR与主分支合并

仓库主人可以在github网站上pull request处同意PR，合并功能已经完成的分支。

```
# 查看分支
git branch              # 查看本地分支
git branch -r           # 查看远程分支
git branch -a           # 查看所有分支

# 创建分支
git branch <branch-name>

# 切换分支
git checkout <branch-name>
git switch <branch-name>    # Git 2.23+ 新命令

# 删除分支
git branch -d <branch-name> # 安全删除（已合并）
git branch -D <branch-name> # 强制删除（未合并）

# 重命名分支
git branch -m <new-name>
```

5、合并与冲突

```
# 合并分支
git merge <branch-name>

# 中止合并
git merge --abort

# 继续合并（解决冲突后）
git merge --continue

# 解决冲突步骤：
# 1. 编辑冲突文件，删除冲突标记（<<<<<<, =====, >>>>>>）
# 2. 添加解决后的文件：git add <file>
# 3. 完成提交：git commit
```

合并之后进入vim编辑器，输入Esc推出编辑模式，之后输入:wq进行退出。

冲突标记说明：

```
<<<<<< HEAD
当前分支的内容
=====
要合并分支的内容
>>>>>> branch-name
```

6、远程操作

远程仓库默认名称是origin。

```
# 查看远程仓库
git remote
git remote -v          # 查看详情，查看远程仓库

# 添加远程仓库，可以只写url，name默认是origin。url=XXX.git
git remote add <name> <url>

# 删除远程仓库
git remote remove <name>

# 拉取远程数据
git fetch <remote>      # 仅拉取，不合并
git pull <remote>       # 拉取并合并

# 推送数据
git push <remote> <branch>
git push -u origin <branch>  # 设置上游分支，push到远程哪个分支上

# 删除远程分支
git push origin --delete <branch-name>
```

远程仓库的分支是什么意思，如何合并，如何创建？

远程仓库分支是托管在远程服务器（如 GitHub、GitLab、Gitee 等）上的分支。

它通常以 `origin/<分支名>` 的形式在你的本地仓库中显示（`origin` 是默认的远程仓库别名，可自定义）。

例如：

- 本地分支：`feature-login`
- 对应的远程分支：`origin/feature-login`

pull request简单例子：

(1)、在本地同步远程仓库，`git remote add`，`git fetch origin+git merge`同步/`git pull`同步。

(2)、创建新分支并进入（例如debug），修改代码，之后`git add`和`git commit`，在debug分支下push到远程仓库，此时远程仓库就多了一个debug分支。（首次push需要-u，建立连接）

`git push` 远程仓库名 本地仓库分支名（远程仓库会创建一个同名的分支）

`git push` 远程仓库名 本地仓库分支名：远程仓库分支名

```
$ Bash

1 # 创建新分支
2 git checkout -b debug
3
4 # 修改代码...
5 git add .
6 git commit -m "修复 bug"
7
8 # 推送到远程
9 git push -u origin debug
```

(3)、可以继续对这个分支push代码，功能完成后。创建PR。

步骤 (3)：继续开发并创建 PR

```
$ Bash

1 # 继续修改，多次提交
2 git add .
3 git commit -m "添加测试"
4
5 git add .
6 git commit -m "优化代码"
7
8 # 推送（因为有关联，可以直接 git push）
9 git push
10
11 # 功能完成后，在 GitHub 上创建 PR
```


在 GitHub 上操作:

PlainText

- 1 1. 打开仓库页面
- 2 2. 点击 "Pull requests" → "New pull request"
- 3 3. 选择:
 - 4 base: master ← 要合并到哪个分支
 - 5 compare: debug ← 要合并哪个分支
- 6 4. 填写 PR 信息:
 - 7 - 标题: 修复 XXX bug
 - 8 - 描述: 修改了哪些文件, 解决了什么问题
- 9 5. 点击 "Create pull request"

push到其它分支, 申请PR合并, 需要经过审核。

push到他人仓库的master分支, 一般是不行的, 只有仓库主人可以push master 分支。

7、撤销与重置

```
# 撤销工作区修改
git checkout -- <file>
git restore <file> # Git 2.23+ 新命令

# 撤销暂存区修改
git reset HEAD <file>
git restore --staged <file> # Git 2.23+ 新命令

# 撤销提交
git reset --soft HEAD~1 # 撤销提交, 保留更改到暂存区
git reset --mixed HEAD~1 # 撤销提交, 保留更改到工作区 (默认)
git reset --hard HEAD~1 # 完全撤销 (危险!)

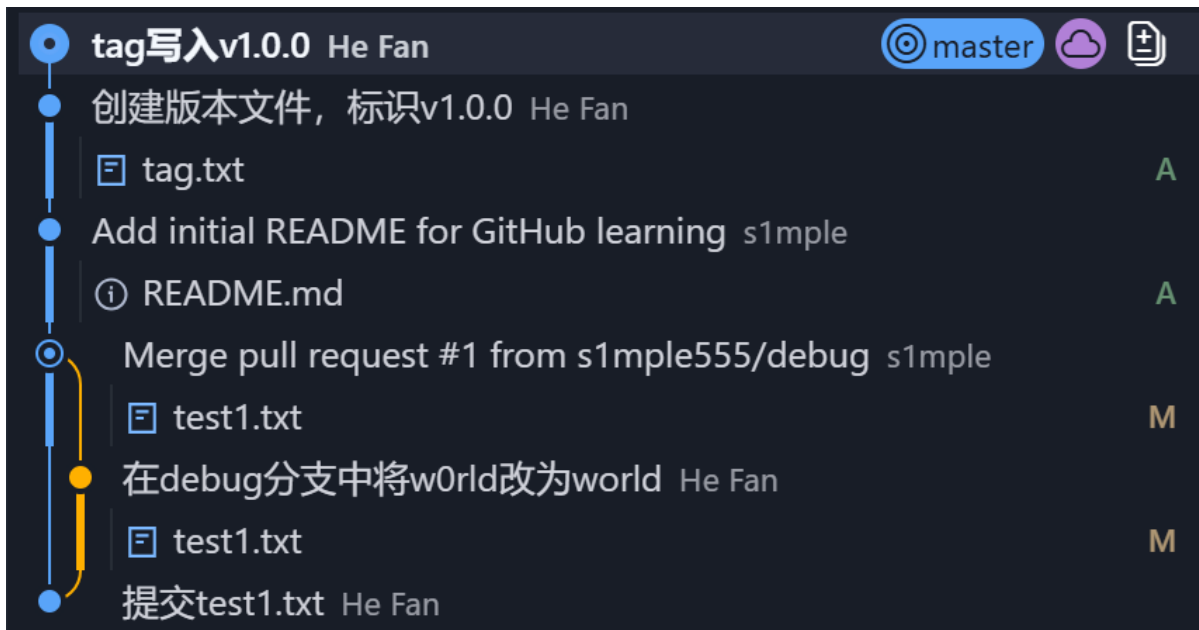
# 修改最后一次提交
git commit --amend # 修改提交信息或添加遗漏文件

# 恢复丢失的提交
git reflog # 查看所有操作历史
git reset --hard <commit-hash> # 恢复到指定提交
```

8、GUI图示解释

蓝色, 本地仓库, git commit创建, 标识目前本地仓库的进度, 若慢于云朵, 说明需要pull, 若快于则需要push。

如果没有云朵, 说明没有建立连接。git push -u origin master, 加上参数-u即可。



常见问题解决

1. 合并冲突解决流程

```
# 发生冲突时
git merge <branch>
# 编辑冲突文件, 解决冲突
git add <resolved-file>
git commit
```

2. 放弃本地所有更改

```
git reset --hard HEAD
git clean -fd
```

3. 查看谁修改了某行代码

```
git blame <file>
```

4. 只克隆特定分支

```
git clone -b <branch-name> <repository-url>
```

5. github的push机制

只允许自己push到master分支, 其他人只能够push到其它分支, 提PR。

- 1. 进入你的 GitHub 仓库页面
- 2. 点击 Settings → Branches
- 3. 点击 Add branch protection rule
- 4. 在 Branch name pattern 输入 `master` (或 `main`)
- 5. 勾选保护选项:

常用保护选项:

选项	作用
☒ Require a pull request before merging	禁止直接 push, 必须通过 PR
☒ Require approvals	需要他人审核才能合并
☒ Dismiss stale pull request approvals	新提交会使审核失效
☒ Require status checks to pass	CI/CD 检查必须通过
☒ Require linear history	禁止合并提交, 保持历史线性
☒ Include administrators	管理员也受限制
☒ Lock branch	完全禁止 push (只读)